6 Building knowledge graphs with large language models

This chapter covers

- Transforming an archive into a knowledge graph
- Graph modeling
- Data normalization and cleansing
- Entity resolution
- Analyzing the intellectual network

In the previous chapter, we discussed extracting complex relational knowledge from unstructured data using state-of-the-art machine learning (ML) technologies, including large language models (LLMs). Specifically, we looked at extracting knowledge from the historical typewritten documents of the Rockefeller Archive Center (RAC). As a brief reminder, these documents contain detailed descriptions of conversations between Rockefeller Foundation (RF) program officers and researchers from a wide range of universities and other institutions. The RF used the information collected during these meetings to decide whether to fund research projects. For a detailed description of the RAC use case and the goals of the project, revisit chapter 5.

As highlighted by the mental model in figure 6.1, we'll pick up where we left off: we've extracted the knowledge from the textual documents, and now we'll explore how to transform it into a KG and how to use the KG for the benefit of our organization.

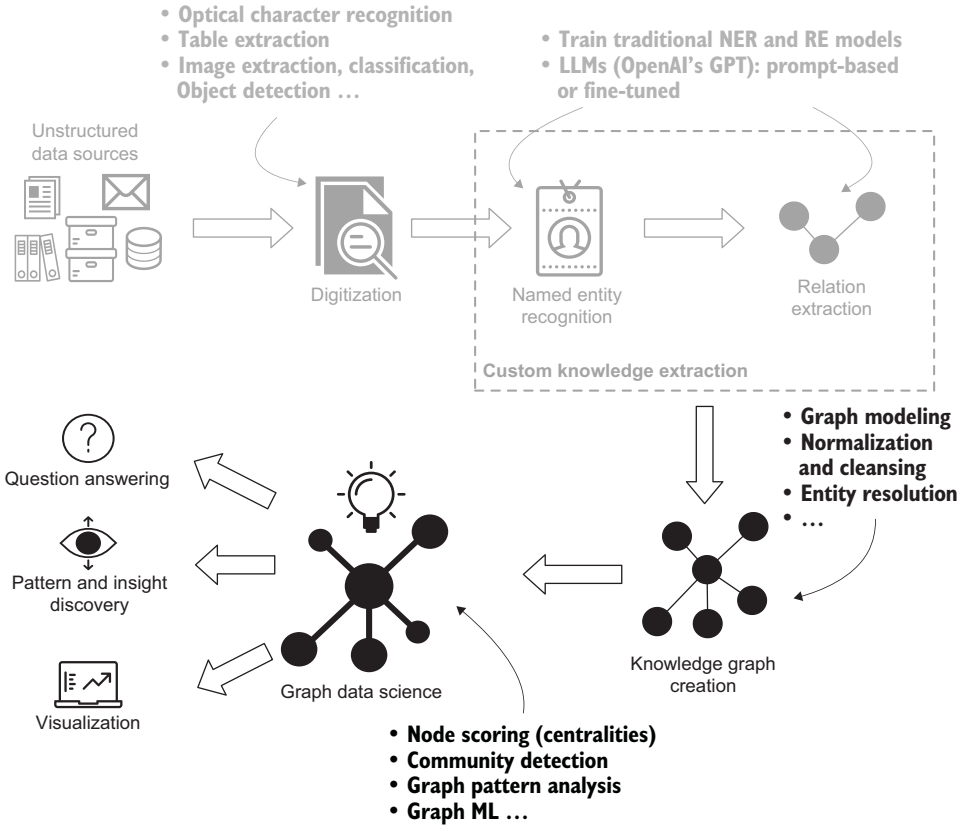

Figure 6.1 Path from domain-specific unstructured textual data toward KG insights. The steps rely on state-of-the-art ML models, such as optical character recognition for document digitization, named entity recognition and relation extraction systems, entity resolution, and GraphML.

6.1 Transforming an archive to a KG

Following is a list of challenges we still need to handle in this project:

- *Analog typewritten documents*—The analog documents need to be scanned and processed by an optical character recognition (OCR) system to produce a digital textual corpus. Various OCR technologies are available, including those from cloud providers like Amazon and Microsoft and the open source Tesseract OCR library.

- *Historical documents*—Many of the research disciplines discussed are no longer pursued. Therefore, there is no hope of compiling a named entity recognition (NER) dictionary or knowledge base comprehensive enough to be used as a reference for disambiguation and entity resolution.
- *Uncommon linguistic conventions*—The program officers had specific writing styles. For example, some of them referred to people and organizations with abbreviations, such as "S." instead of "J. R. Smith" or "U.Cal." instead of "University of California." None of the off-the-shelf coreference models we tested could resolve these shortened names into canonical forms. However, GPT was successful at implicitly performing NER, entity resolution and relation extraction (RE) tasks in response to our prompts.
- *Domain-specific named entities*—We're working with descriptions of research projects in the natural sciences domain. The primary custom-named entities are `Occupations` and include research disciplines, technologies, treatments, and diseases, and they vary in granularity. No traditional NER model is available for these entities (except for diseases). We can however implement a custom ML-based NER and then design, for example, an unsupervised entity resolution system to cluster semantically similar occupations, allowing for further downstream analyses.
- *High relational complexity*—The relational knowledge in these documents is dense and complex. A single page can contain dozens of relevant relations. It is vital to define the RE schema properly so we don't retrieve useless knowledge while potentially missing opportunities to achieve higher accuracy. This is especially important when training RE models by manually labeling large datasets, but also when using the RE schema to guide an LLM for this task.
- *KG normalization, cleansing, entity resolution, and disambiguation*—These tasks will require significant effort. For example, each document may use a different variant of the same name; some conversations have multiple participants who speak about their research in semantically similar, yet different, words; and conversations are often part of chains that take place over months or years. Unfortunately, this kind of analysis is outside of the scope of this book; here we will focus on producing and analyzing interaction networks.
- *Matching/linking unstructured data sources*—Ideally, we'd like to reconcile (match) knowledge extracted from the officer diaries with another data source—board of directors minutes—in a single KG. Doing so would let us identify which conversations resulted in which grants and answer questions such as, "Are there any patterns that typically precede the funding of an idea?" Again, this is outside the scope of this book, but it serves as an illustration of the possibilities that lie ahead when we use unstructured data sources.

Although they may seem daunting, we can call ourselves lucky to live in the LLM era. In the not-so-distant past, we would have had to spend significant resources to build

traditional ML models and strategies to overcome these obstacles. Today, as we shall see in the remainder of this chapter, many of them can be solved with the right choice of a knowledge representation system, an LLM model, and prompt engineering.

6.1.1 Graph modeling

The KG schema of the RAC project is shown in figure 6.2 (simplified for this book). We will focus on one aspect the KG: *influence networks*. These are the relations among people interviewed by RF representatives, such as who talked with whom about who else.

We decided to design the graph in three layers:

- *Document-level layer*—The result of initial data ingestion. Each diary file is represented as a `File` node with properties such as file name, location, and author; and each `Page` node, as result of extraction from each file by the OCR model, with properties such as the final clean text of the page.
- *Metagraph layer*—Unmerged GPT entities (`Entity` nodes) and relations among them, as well as their linkage to the original page from which they were extracted.
- *KG layer*—Final, normalized, cleansed resolved entities (`Person`, `Title`, `Organization`, `Occupation`) and relations among them (`WORKS_ON`, `WORKS_FOR`, etc.).

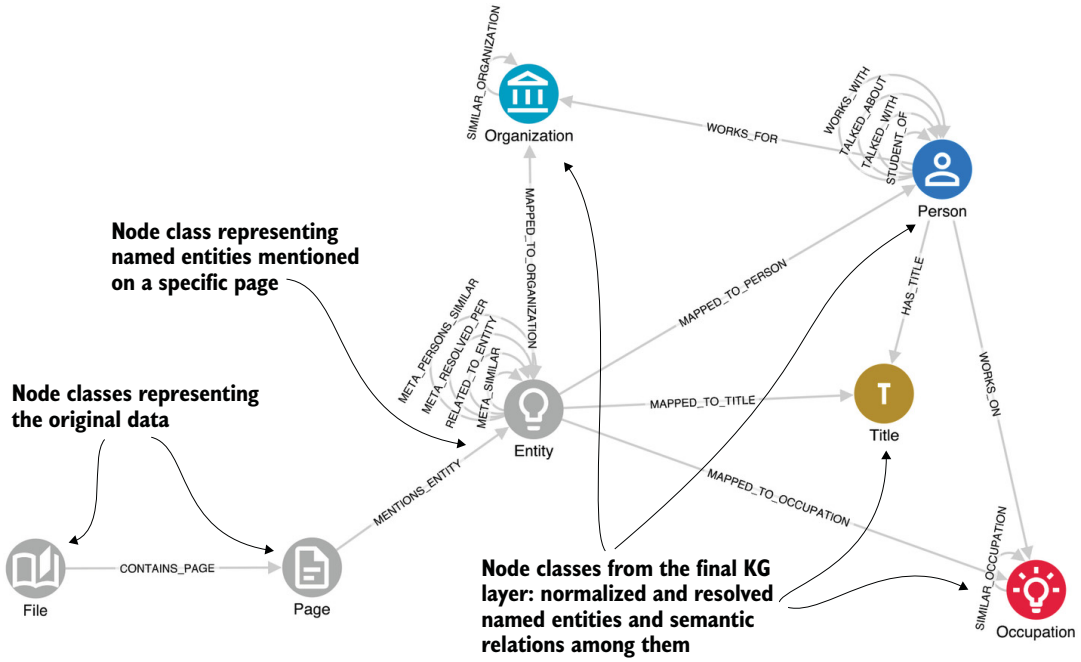

Figure 6.2 Simplified KG schema of the Rockefeller Archive Center project

As we've discussed, an LLM does its best to produce the desired output, but it cannot be used to produce a KG directly. First we need to perform the normalization and entity resolution steps, which are enabled by this schema.

6.1.2 Creating a metagraph

Let's briefly discuss how to create the metagraph. When dealing with texts longer than a few pages, the first step is to define a chunking strategy to avoid running into the max tokens limit or deteriorating processing quality for very long texts. Here we show the simplest approach: splitting by page.

> **NOTE** We had more than 10,000 pages for this project, but for this example, we selected a subset of 150 pages of Warren Weaver's diary. You can find the OCRed dataset and full ingestion code in the book's code repository. The documents are typewritten and date from 1939, so the digitization process may have misspelled some entity names.

After each page is processed, we create (not merge!) all the identified entity mentions (nodes called `Entity` in the schema with property `name`), link them to their page, and create their relations extracted from the given page (called `RELATED_TO_ENTITY` with the `type` property representing the relation class). This allows us to do normalization and entity resolution based on knowledge about each entity mention. Then, when creating the final KG, we can easily aggregate all the knowledge extracted about all the resolved entity mentions across all the pages while preserving the underlying information about the origin of these entities and relations. That will allow us to design the graph visualization platform in an easily explainable manner by being able to show on demand the original text snippets from which the selected entities and relations were identified. It is also useful for data scientists maintaining the KG system to be able to easily track the origin of nodes and relationships in the graph and, for example, fine-tune entity resolution as needed, because the final KG can be re-created anytime from the metagraph.

6.1.3 Normalization and cleansing

Once the metagraph is created, we can inspect statistics such as top entities per class, top relation classes, and so on. This gives us a quick overview of the structure of the knowledge. Plus, if we notice an opportunity to increase future graph connectivity, we'll implement it: for example, lowercasing entity names if case is irrelevant (such as `Occupation`s). This type of normalization is important for ensuring data (KG) consistency and efficient integration across documents. Similarly, GPT occasionally included a person's title as part of their name, even though it was instructed to treat them separately, so we implemented a cleansing strategy that strips irrelevant tokens from people's names to avoid having the same person represented in the KG by multiple nodes (once with and once without the title). The Cypher queries to fix these cases are as follows.

Listing 6.1 Normalizing people and occupations

```
from neo4j import GraphDatabase

URI = "bolt://localhost:7687"
AUTH = ("neo4j", "password")
NEO4J_DB = "neo4j"

REMOVE_TITLES = ["dr.", "prof.", "dean", "president", "pres.", "sir",
➥"mr.", "mrs."]
QUERY_NORM_PERSONS = """                          <-- Cleans Person names: removes titles/degrees
➥ MATCH (e:Entity {label: "Person"})
➥ WITH e, CASE WHEN ANY(title IN $remove_titles WHERE toLower(e.name)
➥ STARTS WITH title) THEN apoc.text.join(split(e.name, " ")[1..], " ")
➥ ELSE e.name END AS name
➥ SET e.name_normalized = name
➥"""

QUERY_NORM_OCCUPATIONS = """                      <-- Lowercases Occupation entity names
➥MATCH (e:Entity {label: "Occupation"})
➥ET e.name_normalized = toLower(e.name)
➥"""

if __name__ == "__main__":
  with GraphDatabase.driver(URI, auth=AUTH) as driver:      <-- Executes the queries
    with driver.session(database=NEO4J_DB) as session:
      print("Normalizing Person names")
      session.run(QUERY_NORM_PERSONS, remove_titles=REMOVE_TITLES)
      print("Normalizing Occupations")
      session.run(QUERY_NORM_OCCUPATIONS)
```

We create new node properties called `name_normalized`, which will be later used instead of unnormalized `name` properties when linking to final nodes in the KG layer.

6.1.4 Graph-based entity resolution

The generative nature of LLMs helps produce cleaner, more accurate KGs. One reason is that, unlike traditional NER and RE models, when properly guided by the prompt or fine-tuning, they return only full, clean entity names. Think of it as coreference resolution, a key task in the standard NLP pipelines, being performed implicitly, thus reducing the need for entity resolution.

Reducing, but not removing. We still need to be able to resolve entities across documents: is "Eleanor Smith" in one document the same person as "E. Smith" in another? A vital part of any KG creation process is thus entity resolution, or even entity disambiguation: linking each subject to a concrete concept in a knowledge base.

In this case, we'll take advantage of the graph structure and design a graph-based entity resolution system. A full discussion is out of the scope of this chapter, but we'll outline the general approach and an initial baseline. Most of the entity mentions in the metagraph layer have one or more relations to other mentions, most of which are useful for entity resolution. Think of the `WORKS_FOR` relations: if there is a high level of string similarity between two names ("Eleanor Smith" and "E. Smith"), and both work

for the same university, that's a strong signal. Similarly, it is unlikely that people with identical or very similar names are working on the same research topic. If we can amass multiple signals of this type, our confidence in this resolution grows.

This approach relies on initially linking nodes in the metagraph layer based on their string similarities. We link two nodes with `META_SIMILAR` relationships if they have identical or very similar string representations. We also take advantage of domain knowledge when defining similarity thresholds. For example, we know that a person's name is composed of first name, middle name(s), and surname. Often, middle names are abbreviated or skipped; the same is true for first names. But relying only on surnames would result in lots of false positives. A combination of surname and at least the first name (or its abbreviation) gives us higher confidence that they could be the same person. We can use these considerations to define a set of rules for creating `META_SIMILAR` edges between nodes representing mentions of person names. Similar reasoning can be applied to other entities, such as `Organization`s.

TIP Sometimes names include generic words. For example, the `Organization` names of many foundations include the keyword *Foundation*, so it would be counterproductive to create similarity links among them. It is important to analyze the situation and define a stopword list before creating `SIMILAR` relationships. Different datasets and domains will require adjusted approaches.

Enough hypothesizing: let's examine the concrete case shown in figure 6.3. We see three mentions of nuclear physicist Ernest Lawrence (who won the Nobel Prize for the invention of the cyclotron), on pages 26, 99, and 126: *Ernest Orlando Lawrence*, *Ernest O. Lawrence*, and *Lawrence*.

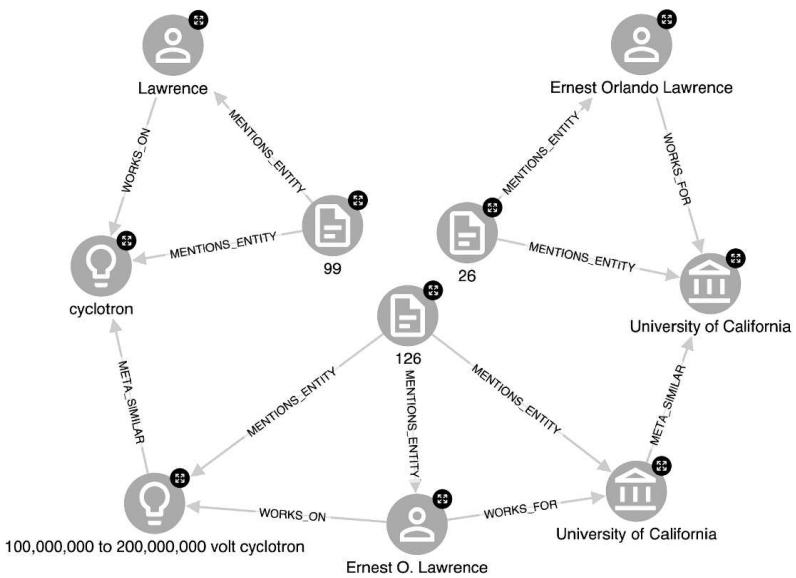

Figure 6.3 Graph-based entity resolution example

How do we know that these three names refer to the same person? There is a strong string similarity based on the rules and patterns we just discussed. We also see that *Ernest Orlando Lawrence* and *Ernest O. Lawrence* are three hops apart, because in both cases they are identified as employees of the University of California. Similarly, we can find a relation to the last mention, *Lawrence*, through the `WORKS_ON` relationships and the similarity between the `Occupations` *cyclotron* and *100,000,000 to 200,000,000 volt cyclotron*. But notice that *Ernest Orlando Lawrence* and *Lawrence* are six hops apart: this kind of traversal would be much harder in a relational database.

Would you like an even more "graphy" approach? How about taking advantage of the intellectual network relations—`TALKED_ABOUT` and `TALKED_WITH`—and running graph community detection algorithms, such as Louvain, to identify clusters of people who interact or, in general, are connected? The reasoning behind this is that *John Doe* working on *maritime research* in *Antarctica* will probably be part of a different interaction network than *John Doe* specializing in *cosmology*. Membership in communities provides another signal to help us decide whether they are the same person.

How about adding ML? One obvious option is to design a semantic similarity approach to identify similar `Occupations`. We can create `META_SIMILAR` links even when there is zero string similarity, but with topics that are highly related: for example, *fertility* and *human ovulation*. A frequent choice is to use embeddings provided by GPT and cluster them based on their similarities (we achieved very good results with an agglomerative clustering approach).

There are many options for graph-based entity resolution; we've just scratched the surface. To conclude our baseline approach for resolving people, we need these last few ingredients:

- Create `META_PERSONS_SIMILAR` relationships among person mentions that comply with the criteria we've defined (string similarity and relatedness through RE outputs).
- Run the weakly connected components (WCC) algorithm on the `META_PERSONS_SIMILAR` metagraph to identify groups of mentions that should be resolved to the same KG entity (final graph layer).
- For each WCC group, select a common name to represent it: we choose the longest one (in the previous example, that's *Ernest Orlando Lawrence*).
- Create the final KG layer with fully resolved entities.

These tasks are straightforward; see our code repository for the full flow.

6.2 Intellectual network analysis: The value of graphs

We've finally reached our goal: a KG. What now? How can we use it? At this point, the general value of KGs should be clear, so we'll explore an analytical aspect: graph data science.

Certain parts of the KG are suitable for graph analytics, especially the *intellectual network* of people (scientists) formed by `TALKED_ABOUT`, `TALKED_WITH`, `WORKS_WITH`, and `STUDENT_OF` relations. We used Neo4j's Graph Data Science library to analyze this

network using graph algorithms such as PageRank, Eigenvector centrality, node degree, and betweenness centrality to identify the following:

- *Influencers*—People who stand out in terms of recommending other people's work
- *Influencees*—Popular targets of other people's referrals (or professional gossip)
- *Bridges*—Those who act as connectors among different communities of people

Different visualization styling options can be designed to help guide us in exploration and analysis.

Figure 6.4 is the largest connected component of the extracted intellectual network with styling based on betweenness centrality: nodes are bigger when more shortest paths among any pair of nodes pass through them. This styling highlights people

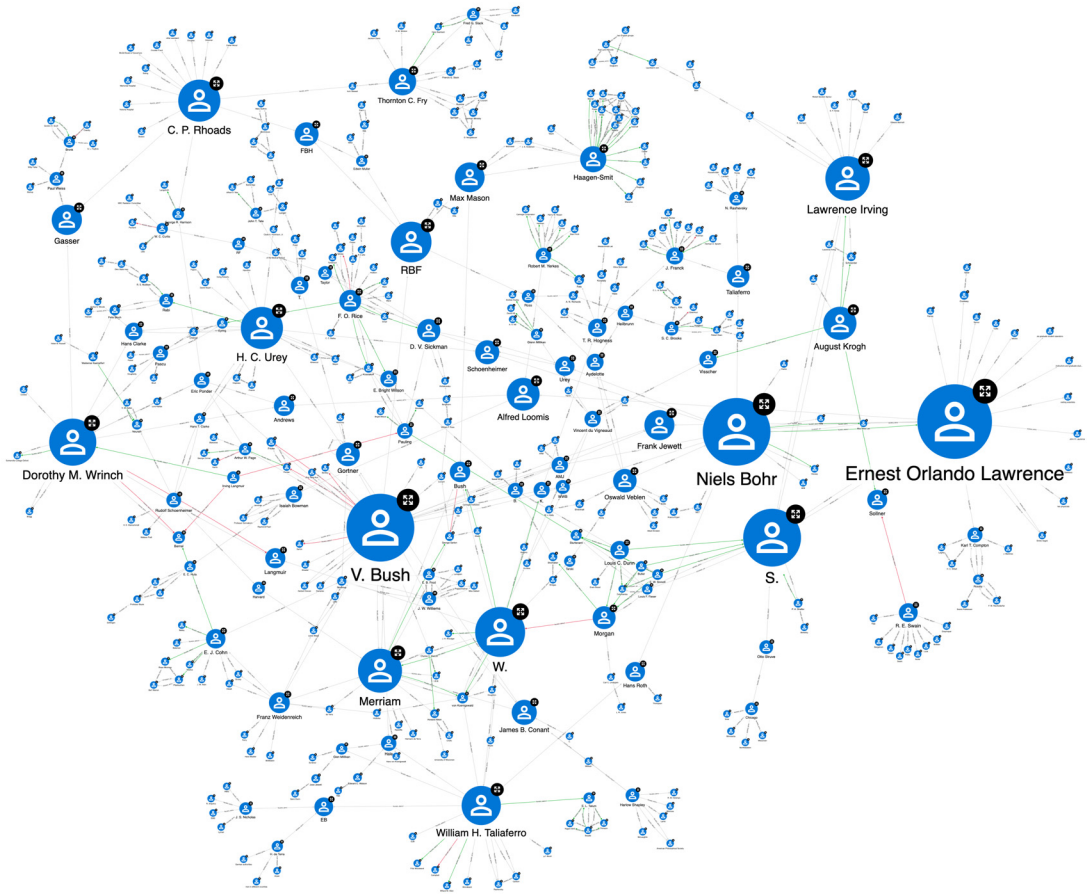

Figure 6.4 The intellectual influence network comprises the relations `TALKED_ABOUT`, `TALKED_WITH`, `WORKS_WITH`, and `STUDENT_OF`. Node styling is based on their betweenness centrality score.

acting as bridges among different subgraphs representing groups of researchers who'd otherwise be very loosely connected or even disconnected. Not surprisingly, famous scientists such as Niels Bohr (the father of atomic physics) and Ernest Lawrence (the inventor of a particle accelerator called cyclotron) are among them; but less famous people are also highlighted, which leads to other, potentially surprising, insights worth investigating.

We can also use our intellectual network to answer more focused questions, such as, "Who played an important role related to the cyclotron research and its funding?" This question can be answered with a simple few-hop query (listing 6.2).

Listing 6.2 Showing the influence network related to cyclotron research

```
MATCH path = ()<-[:WORKS_ON|WORKS_FOR]-(p2:Person)
➥-[:TALKED_ABOUT|TALKED_WITH|WORKS_WITH|STUDENT_OF*1..2]->
(p:Person)-[:WORKS_ON]->()-[:SIMILAR_OCCUPATION*0..1]-(o:Occupation)
WHERE o.name = toLower($occupation) AND
➥NOT ANY(x IN nodes(path1) WHERE x.name = "WW")
RETURN path
```

Within the matched path, we allow up to a two-hop distance between the person working on cyclotron research and some other person. That allows us to explore more complex referral patterns. The graph representation of this influence network of the cyclotron-related research is shown in figure 6.5, along with information such as occupations and university affiliations. The PageRank centrality calculated on the full influence network graph was used to scale the nodes. We see that besides Ernest Lawrence and Niels Bohr, other important people appear nearby, including Harlow Shapley, astronomer and head of Harvard College Observatory, and James B. Conant, organic chemist and 23rd president of Harvard University (Harvard built a cyclotron, which, after secret negotiations with General Leslie Groves, President Conant later sold to the U.S. government for $1 to help the development of the first nuclear bomb).

There is also an element of surprise. Notice the presence of Laurence Irving (with the first name misspelled by GPT as *Lawrence*—a mixup apparently caused by the presence of Ernest Lawrence on the same page), a pioneer in comparative physiology. Is it possible that he played a role in the influence network of the cyclotron invention? That would be unexpected. A closer look reveals that this is an example of failure in the RE task: he should not appear in this subgraph. This is an important reminder that LLMs are not magic; they do make mistakes, and sometimes silly ones. It is important to design a feedback loop in your KG applications so that analysts can validate or invalidate the content of the graph.

Let's conclude with one more example. Imagine that person with experience in a particular domain—in our case, project officer Warren Weaver—leaves their post, and someone new is hired. They're asked to handle a physics research project that spans

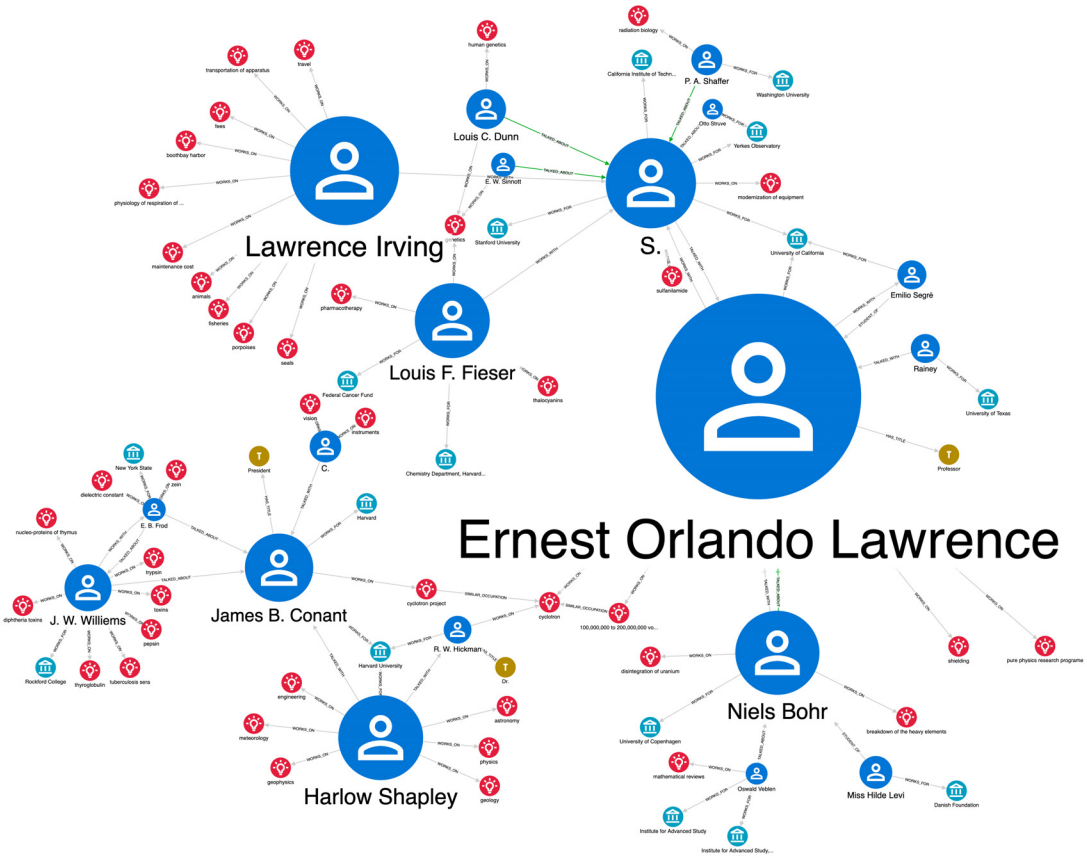

Figure 6.5 Influence network in the two-hop vicinity of the cyclotron research. The `Person` node sizes are based on global PageRank centrality to highlight popular nodes in the graph.

Johns Hopkins University and Harvard University. Which person should they approach first? They need to sound someone out informally, ideally a person with exposure to the physics domain at both Johns Hopkins and Harvard. The question can be answered by inspecting the relevant part of the influence network: we take all employees of both universities (`WORKS_FOR` relations) and search for any connections among them (relations `TALKED_ABOUT`, `TALKED_WITH`, `WORKS_WITH`, and `STUDENT_OF`) with, say, up to three hops, where at least one of the people works on physics research. The result is shown in figure 6.6.

If this were a bigger network, we could run a betweenness centrality algorithm on this subgraph to help us identify the important connectors, but in this case, it is a simple task. Only a handful of people stand out at first glance as useful connectors between the two universities. A good start could be Irving Langmuir (chemist,

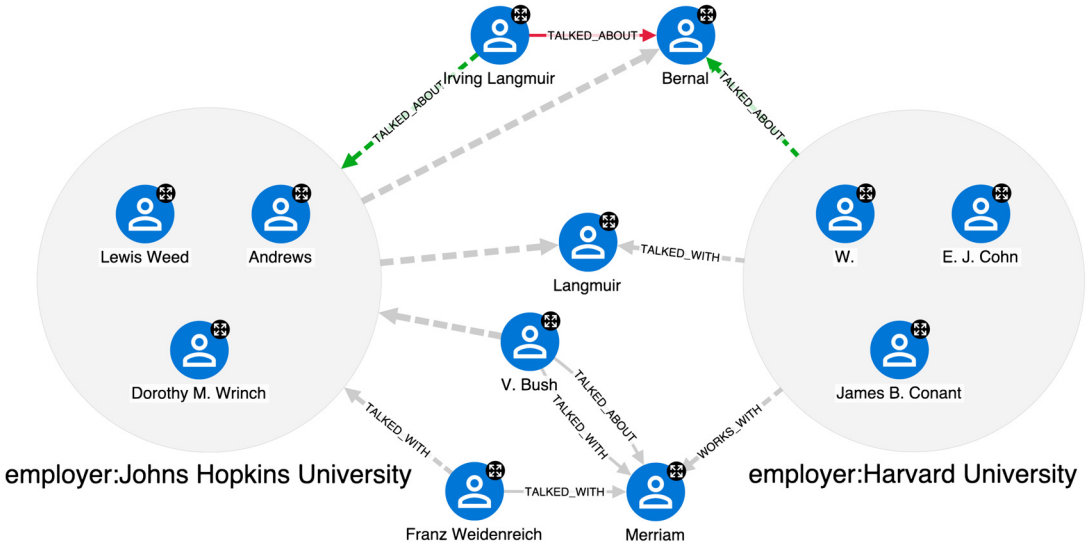

Figure 6.6 Physics research influence network connecting Johns Hopkins University and Harvard University, with no more than three hops between people

physicist, and engineer who won the Nobel Prize in Chemistry in 1932), who talked positively about Dorothy M. Wrinch (who studied insulin and protein structures using X-rays) and who has a direct one-hop link to both universities. Notice that two nodes represent this scientist, *Irving Langmuir* and *Langmuir*, because once his surname was mentioned on a page without additional relations that could be used during entity resolution. Moreover, by examining the properties of `TALKED_ABOUT` relationships that indicate sentiment, we discover that Bernal has a negative attitude toward Dorothy Wrinch, and Irving Langmuir has a negative attitude toward Bernal. To obtain balanced insight, we might want to interview both.

6.3 Next steps in the Rockefeller Archive Center project

The results in the previous section are based on only 150 pages from a much larger data source, but they demonstrate the impressive complexity of the KG. In a full production-quality project, we'd need to do more:

- *Improve knowledge extraction.* More iterations of prompt engineering or fine-tuning the LLM will improve the accuracy of the KG.
- *Handle multipage documents.* Each diary has hundreds of pages, but there is a token limit of how much data we can send to and retrieve from the LLM. In the RAC project, we used ChatGPT-3.5-Turbo to identify the boundaries of individual diary entries, which typically contain fewer than three pages, and process them simultaneously.

- *Perform entity resolution.* We can improve the baseline approach presented in this chapter, expand it to other entities, and complement it with entity disambiguation against WikiData or a similar knowledge base.
- *Add grants.* Mining details about grants awarded by the RF from board of directors minutes and linking the grants to conversations in the diaries will let us answer questions such as, "Do granted projects tend to run through recommendations of influential scientists or previous grantees?"
- *Perform entity resolution of* `Occupation`s. These are named entities with varied granularity: for example, we have *nuclear physics*, *isotopes*, and *heavy nitrogen*, but both isotopes and heavy nitrogen are part of nuclear physics and heavy nitrogen is an isotope of nitrogen. To answer complex questions, we need to be able to link (resolve) them and thus gain access to the entire history of the given topic. We've gotten the best results by creating embeddings of `Occupation`s (using SentenceBERT or GPT) and clustering them using agglomerative hierarchical clustering.
- *Create conversations.* We can create `Conversation` nodes by using high-quality RE and other information: a `Conversation` needs a date, an interviewer, interviewee(s), and a topic (`Occupation`s of the interviewees). Once we have `Conversation`s, we can identify their follow-up chains and link them to grants. Both tasks are achievable thanks to the unsupervised resolution of occupations.

Now the full extent of the RAC project is clear. Bleeding-edge knowledge mining from historical typewritten documents, reconciling unstructured data sources, entity resolution systems, graph modeling, graph data science, complex visualization, and styling—it's challenging, it's tough, and, most importantly, it's fun!

6.4 The value of knowledge graphs in the LLM era

You may wonder why, in the era of super-powerful Large Language Models, we build KGs instead of feeding these models our data and asking questions directly, without intermediate steps. The answer to this question is complex, and this book—our testament to KGs—is the big answer. But we can also provide a briefer answer, related to this chapter and summarized as follows:

- *Explainability*—Applications based on well-designed KGs have a huge advantage in being natively explainable. They give users the tools to inspect and verify the underlying data and reasoning, and they can be configured to handle conflicting sources of information while providing the entire chain of "thought" when required.
- *Demystification, or de-black-boxing*—People often view advanced ML models as black boxes that they are supposed to blindly trust. If we simply fine-tuned an LLM on our dataset and asked it questions, we would have no way to assess the confidence of the responses. And could we be sure that the "AI" wouldn't miss a crucial part of the information in our data? Instead, using the language understanding power of these models to extract specific factual information

from documents and produce a KG gives us confidence in the generated insights.

- *Democratization*—LLMs are massive beasts that are expensive to train and fine-tune. We can think of KGs as one way to democratize their use so that even organizations without massive funds can profit from them: we can use the expensive model only once to produce the KG, which we will then use for a long time (perhaps with occasional inexpensive batch updates) for downstream tasks and analyses.
- *Explorability*—Graphs let users view and touch their (relational) data from new angles. They provide global views as well as drill-down investigations. And KG visualization and explorability inspire people to hypothesize and then verify or disprove their theories.
- *Advanced analytics*—Perhaps most importantly, KGs empower data scientists and analysts to perform downstream graph-based analyses and ML while giving them full control over the generation of answers to user questions.

Summary

- Knowledge Graph schema design, a.k.a. graph modeling, ensures that information is stored optimally for the use case. A well-designed schema simplifies KG creation, entity and relation cleansing, normalization, and resolution, and ensures efficient downstream analyses and insight discovery.
- Extracting high-quality entity relations from textual documents helps us design an unsupervised graph-based entity resolution approach.
- Various graph data science and graph ML techniques can help us analyze patterns and draw insights from KGs without relying too heavily on black-box tools such as LLMs.